

Project Documentation

A Mobile-First Marketplace Connecting Ghanaian Smallholder Farmers to Buyers, Market Prices, and Agricultural Advisory Services

The master narrative of this capstone project — background, problem, objectives, stakeholders, design rationale, challenges, limitations, and conclusion — cross-referencing five companion technical documents for full detail rather than repeating them wholesale.

Course	CSCD602 — Advanced Software Engineering, Capstone Project
Institution	University of Ghana — Department of Computer Science
Group	Angry Nerds
Project title	FarmConnect Ghana
Document version	1.0
Date	12 August 2026
Individual submission by	George Boadu Junior — 22427354

Group Information

#	Group member	Student ID	Major contribution
1	George Boadu Junior (this submission)	22427354	Requirements engineering & SRS authorship, system architecture & design, full-stack implementation (API + PWA), test authoring/execution, deployment engineering, and project documentation.
2	Marvin Kabutey	22425067	
3	Obed Borley Obuadey	22424177	
4	Richmond Gakpetor	22424720	
5	Isichei Phelim	22424142	
6	Edem Afflu	22424139	
7	Emmanuel Okyere Gyateng	22424179	
8	Edinam Ahadzie	22424539	
9	Desmond Techie	22424555	
10	Irene Hamilton	22424758	
11	Bernard Tetteh Djangbah	22424329	

How this documentation package is organised

To keep each deliverable focused and to avoid a single unmanageable document, the required content is split across seven PDFs. This document is the narrative spine; the other six hold the deep technical/operational detail and are cross-referenced throughout rather than duplicated.

- **Project_Documentation.pdf** (this document) — background, problem, objectives, stakeholders, design rationale, implementation summary, challenges, limitations, conclusion, references, appendices.
- **SRS.pdf** — full IEEE-830 Software Requirements Specification, with a v1.0→delivered-system reconciliation section.
- **Design_Documentation.pdf** — architecture, use-case/class/sequence/state diagrams, ER diagram, database design, UI wireframes, technology architecture.

- **Testing_Report.pdf** — real automated + manual test evidence, defect log, UAT checklist.
- **User_Manual.pdf** — step-by-step guide for Farmers, Buyers, and Administrators, plus the SMS command reference.
- **Maintenance_and_Evolution.pdf** — maintenance categories, bug-fixing process, dependency/security update policy, scalability plan, and the future roadmap.
- **Deployment_Guide.pdf** — the numbered runbook actually used to put the live system online on a \$0/month hosting stack.

Table of Contents

1	Introduction & Background
2	Problem Statement
3	Aim & Objectives
4	Stakeholder Analysis
5	Requirements Gathering & Analysis
6	Software Requirements Specification
7	System Analysis & Design
8	Architecture
9	Database / Data Design
10	User-Interface Design
11	Implementation
12	Testing & Quality Assurance
13	Deployment
14	User Manual
15	System Administration & Maintenance Guide
16	Security Considerations
17	Challenges & Solutions
18	Limitations
19	Maintenance Strategy
20	Future System Evolution / Enhancements
21	Conclusion
22	References
23	Appendices

1. Introduction & Background

Ghana has over 2.7 million smallholder farmers, most of whom farm fewer than 2 hectares and sell into informal, fragmented supply chains with little price transparency. FarmConnect Ghana is a mobile-first web platform built to close that gap directly: it lets a farmer publish what they have to sell, lets a buyer find it and pay for it, and wraps that core transaction in the supporting infrastructure a real rural market needs — live price information, an AI-powered agricultural advisor, a feature-phone SMS channel for users without a smartphone, and cooperative/trust/dispute-resolution mechanisms.

This project was undertaken as the CSCD602 Advanced Software Engineering capstone, carried through the complete software engineering lifecycle — requirements engineering, system analysis and design, implementation, testing, deployment, and documentation — by Group Angry Nerds, University of Ghana, Department of Computer Science. It is not a mock-up: every screen, endpoint, and workflow described in this documentation set exists in running, tested code (see the Testing Report) and is reachable at the live application URL in [Links.txt/README](#).

2. Problem Statement

- **No direct market access.** Smallholder farmers typically sell through middlemen who, by the project's own market research (cited in the original SRS), capture up to 60% of the potential profit margin.
- **No price transparency.** Without a reliable, current view of commodity prices, farmers cannot tell whether an offered price is fair, weakening their negotiating position further.
- **Post-harvest loss.** An estimated 30–40% of harvested produce spoils before it reaches a buyer, a direct consequence of slow, informal, geographically limited sales channels.
- **Digital exclusion.** Existing digital agriculture platforms are largely designed for smartphone-first, English-only, high-connectivity users — excluding exactly the population (feature-phone users, Twi speakers, low-end Android devices, intermittent 3G) this project targets.
- **No trusted way to transact remotely.** A farmer and a buyer who have never met have no shared trust signal and no safe way to hand over money for produce that hasn't been seen yet — a real barrier to any farmer-to-buyer digital marketplace in this context, not a hypothetical one.

3. Aim & Objectives

Aim: to design, build, test, and deploy a complete, functional, low-cost digital marketplace that connects Ghanaian smallholder farmers directly to buyers, market price information, and agricultural advisory services — accessible on both smartphones and basic feature phones.

Objectives

1. Enable farmers to list produce (crop, quantity, price, harvest date, photos, location) and buyers to discover and order it, without a middleman.
2. Provide a safe, zero-custody payment hand-off via Mobile Money, with a human-verifiable confirmation step and an administrator-arbitrated dispute path.
3. Surface live, per-crop market prices so farmers can negotiate from a position of information, not guesswork.
4. Provide an AI-powered advisory channel for crop, pest, and weather guidance, including photo-based pest identification.
5. Support users with no smartphone or data plan via a parallel SMS command channel covering every critical action.
6. Build trust between strangers transacting for the first time via a mutual rating system and an administrator moderation/dispute layer.
7. Deliver the system bilingually (English/Twi) and accessibly, and deploy it to a publicly reachable, live URL.

4. Stakeholder Analysis

Stakeholder	Interest	How the system serves them
Smallholder farmers	Primary user, seller	Listing creation, price feed, advisory chat, SMS fallback, direct Mobile Money payment with no platform fee.
Buyers (individuals, retailers, wholesalers)	Primary user, purchaser	Search/filter, farmer trust signals, transparent order/payment tracking, dispute path if something goes wrong.
Platform administrators	Operate & moderate the platform	Admin Portal — verify/suspend accounts, moderate listings, resolve payment disputes.
Agricultural extension officers / NGOs	Named in the original SRS as potential pilot partners; monitor regional trends	Reserved (<code>EXTENSION</code>) role in the data model, activation planned as Maintenance & Evolution Plan Phase 12 — see §20.
Course instructors / examiners	Assess engineering rigor and completeness	This documentation set, the live deployment, the source repository, and the test evidence in the Testing Report.
Future maintainers	Keep the system running and evolving	The Maintenance & Future Evolution Plan, migration history, and a documented bug-fixing/release process.

5. Requirements Gathering & Analysis

Requirements were gathered by combining (a) published statistics on Ghana's smallholder-farming sector and known post-harvest-loss/middleman-margin figures (cited in SRS §1.4), (b) domain reasoning about the concrete constraints of the target user (feature phones, intermittent connectivity, Twi as a first language for many farmers, no bank/card relationship but near-universal Mobile Money access), and (c) iterative refinement of an AI-assisted first draft (disclosed in full in SRS Appendix A) into a structured IEEE-830 specification, which the team then reviewed against what could realistically be built, tested, and deployed within a single-semester capstone.

The single most important analytical step taken *after* v1.0 of the SRS was written was reconciling it against implementation reality once real engineering constraints appeared — most significantly, that this team could not obtain production Mobile Money merchant/PSP API access, and that no public Ghana Commodity Exchange API exists to integrate with. Rather than silently drop or quietly reinterpret those requirements, every such gap was analysed, a concrete alternative was designed, and the decision and its rationale were written back into the SRS as a dedicated reconciliation section (SRS §9) — the analytical output of this project's requirements work is therefore the full v1.0→v2.0 diff, not just the final list of what got built.

6. Software Requirements Specification

See **SRS.pdf** for the complete document. Summary: 14 functional requirements (FR-01–FR-14) and 13 non-functional requirements across performance, security, usability, reliability, and maintainability, each carrying a Status column that records whether it was delivered as specified, delivered with an approved and documented deviation, or explicitly deferred. Section 9 of the SRS is a dedicated reconciliation between the original v1.0 baseline and the delivered v2.0 system — the single most load-bearing section for understanding *why* the system looks the way it does.

7. System Analysis & Design

See **Design_Documentation.pdf** for the complete set of diagrams. In summary, the system was analysed and designed as a three-tier architecture (PWA client / REST API / Postgres+Redis data layer, plus four external service integrations), with:

- A **use-case diagram** covering 8 use cases across 4 actors (Farmer, Buyer, Administrator, and the reserved Extension Officer role).
- A **class diagram** and a full **entity-relationship diagram** covering all 10 database tables, their fields, and cardinalities.
- Three **sequence diagrams** (authentication/token-refresh, the order/payment lifecycle including the dispute branch, and the AI advisory flow).
- A **state diagram** for the `Order` status machine — the most behaviourally important piece of the whole system.
- A **component diagram** showing the module-per-domain backend layout and its dependency on the shared validation/enum package.
- Low-fidelity **UI wireframes** of the actual delivered screens.

8. Architecture

Client React PWA (Vite, TS) — Farmer/Buyer/Admin UI, bilingual, offline-caching	API Node.js/Express (TS) — 11 domain modules, JWT auth, zod validation	Data PostgreSQL 16 (Prisma) + Redis 7	External Gemini · OpenWeatherMap · GiantSMS · Africa's Talking
---	--	--	---

Full deployment topology, provider rationale, and the technology-choice justification table are in Design_Documentation.pdf §2 and §10; the live-hosting version of this same architecture (Vercel/Render/Supabase/Upstash) is in Deployment_Guide.pdf.

9. Database / Data Design

PostgreSQL 16 via Prisma ORM, 10 tables (`users` , `listings` , `orders` , `ratings` , `coop_groups` , `coop_members` , `chat_messages` , `notifications` , `price_snapshots` , `sms_inbound_logs`), built up over 7 tracked migrations. Data integrity is enforced at the database level wherever possible (unique constraints for "one co-op per farmer" and "one rating per rater per order," not just application checks). Full field-level schema, indexes, and the ER diagram are in [Design_Documentation.pdf §6](#).

10. User-Interface Design

Mobile-first, bilingual (English/Twi), with a bottom-tab navigation that differs by role (Farmer: Home/Listings/Orders/Prices/Profile; Buyer: Market/Orders/Advisor/Prices/Profile) and a separate, desktop-oriented Admin Portal with no bottom navigation. Colour palette, screen names, and copy were carried forward from an early Figma-Make UX prototype (SRS Appendix B) into the real implementation, though the prototype itself was never pixel-matched. See [Design_Documentation.pdf](#) §9 for wireframes of every major screen, and [User_Manual.pdf](#) for what each screen actually does from a user's point of view.

11. Implementation

11.1 Stack

Frontend	Vite + React 18 + TypeScript, Tailwind CSS, i18next, vite-plugin-pwa
Backend	Node.js + Express 4 + TypeScript, Prisma ORM, ioredis, Zod validation, pino logging
Data	PostgreSQL 16, Redis 7
Monorepo	npm workspaces — apps/web , apps/api , packages/shared

11.2 What was actually built

All 7 planned implementation phases are complete: (0) scaffolding, (1) phone+OTP auth with role selection, (2) listings + marketplace search, (3) orders + manual Mobile Money reconciliation, (4) AI advisory chatbot with photo pest-ID, (5) live price feed + notifications + SMS fallback, (6) co-ops + admin/moderation + ratings, (7) PWA offline caching + full bilingual pass + accessibility pass. This is not a partial or prototype build — every use case in §7 is backed by working, tested code (Testing Report §3/§5).

11.3 Code organisation

The backend is organised into 11 domain modules (auth, users, listings, orders, prices, advisory, notifications, co-ops, admin, sms-gateway, uploads), each with its own routes/controller/service files, sharing common middleware (`requireAuth` , `requireRole` , `validateBody/Query`) and a single Prisma client. The frontend mirrors this with a page-per-screen structure and a thin, centralised API client that handles token refresh transparently. Both apps depend on `packages/shared` for every enum and Zod validation schema, so the two apps cannot silently drift apart on what a valid request/role/status looks like.

11.4 Error handling & validation

Every request body/query is validated against a Zod schema before it reaches business logic (400 on failure, with field-level error detail); every domain error (e.g. "not your listing," "listing already pending," "momo not set up") is a typed error class mapped to the correct HTTP status by a single `handleKnownErrors` helper, rather than ad hoc `try/catch` scattered per route.

12. Testing & Quality Assurance

See **Testing_Report.pdf** for the complete, evidence-based report. Headline results, all captured on 12 August 2026 against a live Postgres+Redis instance:

- **69/69 automated tests passing** across 17 test files (unit + integration) in all three workspaces.
- **11/11 manual system/security smoke tests passing** against the actually-running API (auth guards, role guards, single-use OTP, input validation).
- **All 7 use cases (UC-01–UC-07) traced to passing automated and manual tests.**
- **One real defect found and fixed during this submission cycle** (a mislinter-but-actually-clean codebase, traced to an ESLint config gap — full before/after evidence in the report) and one open, scoped performance finding with a concrete recommended fix.

13. Deployment

See **Deployment_Guide.pdf** for the full numbered runbook, and **Links.txt/README** (submitted alongside this package) for the actual live URLs and credentials. The system is deployed on an entirely free-forever hosting stack chosen specifically to avoid time-limited trial credits:

Vercel Web PWA	Render API (Node/Express)	Supabase PostgreSQL 16	Upstash Redis
--------------------------	-------------------------------------	----------------------------------	-------------------------

Deployment configuration is checked into the repository (`vercel.json` , `render.yaml`) so both platforms redeploy automatically on every push to `main` — deployment is not a manual, undocumented, one-off act.

14. User Manual

See **User_Manual.pdf** for the complete guide, covering: getting started and account creation (no password, ever — phone+OTP only); the full Farmer guide (Mobile Money setup, listing produce, handling an order and getting paid, disputes, co-ops, ratings, prices, the AI advisor); the full Buyer guide (searching, ordering, paying, confirming delivery); the Administrator guide (verifying/suspending users, moderating listings, resolving disputes); the complete SMS command reference for feature-phone users; and a troubleshooting/FAQ section.

15. System Administration & Maintenance Guide

See **Maintenance_and_Evolution.pdf** §2–§5 for the complete operational guide: the bug-fixing process and severity/response targets, version-control and release strategy (Git/GitHub, migration-based schema changes, redeploy-to-rollback), the dependency-update policy, and the security-update process (secret rotation, CVE handling, and two concrete recommended hardening steps — httpOnly refresh-token cookies and IP-based rate limiting — that are not yet implemented but are scoped and ready to pick up).

Administrator account provisioning specifically: there is no self-service admin sign-up (SRS §9.3); an administrator is created by running `npx tsx apps/api/prisma/seed.ts --phone <number> --name <name>` against the production database (Deployment_Guide.pdf Step 8), after which that phone number authenticates through the ordinary phone+OTP flow like any other user.

16. Security Considerations

Area	Position taken
Authentication	Phone + OTP (no passwords to leak or reuse); OTP hashed at rest (bcrypt) with a 5-minute TTL and single-use deletion; 20/hour/phone rate limit against abuse.
Session management	Short-lived (12h) JWT access tokens; 30-day refresh tokens that are single-use and rotate on every refresh via a Redis allowlist — logout genuinely revokes a session, which a bare stateless JWT cannot do on its own.
Authorisation	Every route explicitly declares its required role (<code>requireRole</code>); ownership checks (e.g. "is this your listing/order?") are enforced in the service layer, not assumed from the UI. Verified by manual probing in Testing Report §4.
Payment security	The platform never stores or transmits a Mobile Money PIN, card number, or wallet credential — by construction, not by policy (SRS §9.1/NFR-S3).
Input validation	Every request body/query validated against a Zod schema before touching business logic (SRS NFR-S2-adjacent, demonstrated in Testing Report §4 test 11).
Known residual risk	Access/refresh tokens are stored in browser <code>localStorage</code> , not <code>httpOnly</code> cookies — a documented XSS-exposure trade-off (SRS §9, Design Documentation "cross-cutting concerns") with a scoped remediation plan in <code>Maintenance_and_Evolution.pdf</code> §5.
Regulatory posture	By never taking custody of funds, FarmConnect avoids acting as a payment service provider under Bank of Ghana e-money rules (SRS §6.1) — a deliberate design choice, not an oversight.

17. Challenges & Solutions

Challenge	Solution adopted
No realistic path to production Mobile Money merchant/PSP API access within a single-semester capstone.	Replaced automated gateway escrow with a manual, human-verified reconciliation workflow that never takes custody of funds — safer regulatory posture, achievable in the available time (SRS §9.1).
No public, developer-accessible Ghana Commodity Exchange price API exists.	Built a self-contained, realistically-behaving simulated price feed behind a thin service boundary, so a real integration can be swapped in later without touching the UI or the rest of the API (SRS §9.2).
The SRS gave ratings, co-ops, and admin/moderation one sentence of intent each, with no workflow or data model.	Designed each from scratch during implementation, with the reasoning for every non-obvious choice (one co-op per farmer, leadership succession rule, no self-service admin signup) recorded in Design_Documentation.pdf §11, not left implicit in code.
Serving both smartphone and feature-phone users from one system.	A parallel SMS command grammar (§Testing Report UC coverage; User_Manual.pdf §5) covering every critical action, sharing the same underlying data model and business logic as the app.
Verifying the system actually works, not just believing it does.	Layered testing strategy (unit → integration against a live database → manual smoke tests against the actually-running server) — see Testing_Report.pdf §1, and the deliberate choice to run tests against real Postgres/Redis rather than mocks, since several real defect classes only appear against real database/cache behaviour.

18. Limitations

Stated plainly, as good engineering practice requires:

- Payments are a manual reconciliation process, not an automated escrow — trust and diligence from both parties still matter, backstopped by (but not eliminated by) the dispute-resolution workflow.
- Market prices are a realistic simulation, not sourced from a real exchange, because no such public API exists to integrate with.
- NFR-P2 (10,000 concurrent users) and NFR-R1 (99.5% uptime) are architecturally plausible but not empirically load-tested at this submission's scale (Testing Report §11).
- Frontend automated test coverage is thin relative to the backend (2 component test files versus 14 integration test files) — the backend carries essentially all business logic, but page-level UI regression coverage is a known gap (Testing Report §11).
- The `EXTENSION` (Agricultural Extension Officer) role named in the original SRS is reserved in the data model but has no implemented screens yet.
- Render's free-tier web service sleeps after ~15 minutes idle, adding a ~30–50s cold-start delay to the first request after a period of inactivity (Deployment_Guide.pdf) — acceptable for grading/demo use, not for a real production pilot at scale.

19. Maintenance Strategy

See **Maintenance_and_Evolution.pdf** §1–§6 for the complete strategy: all four ISO/IEC 14764 maintenance categories (corrective, adaptive, perfective, preventive) mapped against real, already-identified items in this codebase rather than generic examples; a concrete bug-fixing process with severity-based response targets; a Git/GitHub-based version-control and release strategy where every release is a redeployable commit; a dependency/library update cadence; a security-update process; and a scalability plan naming the specific bottlenecks (search, uploads storage, price-tick/notification fan-out) that would need addressing first under real growth, and what to do about each.

20. Future System Evolution / Enhancements

See **Maintenance_and_Evolution.pdf** §7–§10 for the full roadmap. In brief, five prioritised, independently-shippable future phases: (8) hardening & observability — close the open performance finding, broaden frontend test coverage, add real load testing; (9) real-time & push — replace REST polling with WebSocket/Web Push; (10) logistics & delivery tracking — the domain explicitly excluded from v1 scope; (11) an optional, feature-flagged licensed Mobile Money aggregator for users/regions that outgrow manual reconciliation; (12) activating the reserved Extension Officer role for agricultural staff and NGO partners. AI/emerging-technology enhancements are scoped as extensions of the capability already in production (Gemini + weather context + price history), not speculative new dependencies — proactive weather-risk alerts, price-trend summarisation, a listing-photo-quality assistant, admin fraud-signal ranking, and voice input for search.

21. Conclusion

FarmConnect Ghana set out to connect Ghanaian smallholder farmers directly to buyers, fair price information, and agricultural advisory services, in a form that genuinely fits its target users — including those on feature phones with no data plan. The delivered system meets that aim: every functional requirement in the SRS is either implemented as specified or implemented via a documented, justified deviation; every core use case is covered by both automated tests against a live database and manual verification against the actually-running server; the application is deployed to a live, publicly reachable URL on infrastructure chosen specifically to remain free and accessible through grading and beyond; and every simplification, trade-off, and open item is written down rather than glossed over, with a concrete plan for what comes next. The gap between "software requirements specification" and "shipped, working system" is where most real-world engineering difficulty lives — this project's documentation set is built to make that gap, and how it was closed, fully visible to an examiner.

22. References

- IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.
- ISO/IEC 14764:2006, *Software Engineering — Software Life Cycle Processes — Maintenance*.
- MTN Mobile Money, Telecel Cash, and AirtelTigo Money — public product documentation.
- Google Gemini API documentation — ai.google.dev.
- OpenWeatherMap Current Weather API documentation — openweathermap.org/api.
- Africa's Talking SMS API documentation — africastalking.com.
- GiantSMS API documentation — giantsms.com.
- Prisma ORM documentation — prisma.io/docs.
- Vercel, Render, Supabase, and Upstash platform documentation (see `Deployment_Guide.pdf` for exact links used).
- FarmConnect Ghana project presentation and original SRS, Group Angry Nerds, May 2026.

23. Appendices

Appendix A — AI-Assisted Drafting Disclosure

In accordance with course AI-disclosure requirements: the original v1.0 SRS was drafted with AI assistance (Claude, claude.ai) via three disclosed prompts, reviewed and finalised by the team, then reconciled against the delivered system in v2.0 (full prompt text in SRS Appendix A). The early UX/flow prototype referenced during design was produced via Figma Make (four disclosed prompts, SRS Appendix B). The v2.0 SRS reconciliation, all five other companion documents, and this master document were authored by George Boadu Junior with AI assistance (Claude Code) for structuring, diagramming, and drafting, working directly from the project's actual source code, test output, and configuration — every technical claim in this documentation set was verified against the real repository and, where applicable, a live running instance of the system, not generated from description alone.

Appendix B — Third-Party Services Acknowledgement

Google Gemini	AI advisory chat + vision-based pest identification
OpenWeatherMap	Weather context for the advisory chatbot
GiantSMS	Outbound SMS (OTP, notifications, SMS-command replies)
Africa's Talking	Inbound SMS command webhook
Vercel, Render, Supabase, Upstash	Hosting (web, API, database, cache respectively)
Prisma, Express, React, Vite, Tailwind CSS, i18next, Zod, and the rest of the open-source dependency tree	See <code>package.json</code> / <code>package-lock.json</code> in every workspace for the complete, versioned list

Appendix C — Group "Angry Nerds"

Full roster and per-member contributions are recorded in "Group Information" at the front of this document.